

FRED TALKS

5th October 2026

CONTENTS

Test Coverage Won't Save You

JENN COOPER · 1991 WORDS

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 1426 WORDS

The End of Code Review? Or an Opportunity to Rethink it?

CHRISTIAN KÄSTNER · 2579 WORDS

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · [SOURCE](#)

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

A room with too much electricity.

"Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

previously done. I expect that coding agents will get much better at reviewing their own code during production, so that few of those more shallow and predictable issues remain that traditional code review or static analysis tools might have surfaced.

I think the bigger question is about all the other benefits we got from code review that were not about code quality, like coordination and mentoring. I think the better question to ask is how much those are still needed and how we can get them in forms that are not code review. For example, would better standup meetings or AI summaries of changes be able to replace the awareness benefits of code review? Would deliberate feedback and mentoring mechanisms better support the junior to senior software engineering pipeline than code review? I think stepping back and rethinking a world without code review is a more useful exercise than tweaking existing code review practices.

Zooming out further, I think eventually the big question is really about where and how and how much humans will be in control in some form when producing software. Maybe that control is better focused on requirements, architectural design, test design, and security/safety safeguards, rather than reviewing lines of AI-generated code.

So my main point is to stop asking whether and how code review in its current form can be saved, but to have an open conversation about what we are trying to achieve here and the costs and benefits of alternative models.

Acknowledgment: This model grew out of a conversation with Courtney Miller about whether preserving code review is the right goal for researchers to support. I am no longer sure it is.

Code review is probably on its last legs, and we should focus on how to retain the indirect benefits it provided...

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, [Garry Tan argued](#) that the primary way to keep AI agents on track is 90% test coverage:

└ *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

Lowered review costs from automated code review cannot be viewed in isolation. Whether automated code review is more or less effective (E) than humans, or effective only for particular classes of bugs, is still unsettled. Automation is likely able to go much deeper and be more systematic than a typical human review (closer to Fagan-style inspections with checklists), but it may also miss issues obvious to humans. If automation produces more false positives than human reviewers, it would push production cost (P) back up, though if production is cheap that may not matter much.

Indirect effects of code review, beyond finding bugs, may or may not matter, and they will decide the future of code review. Code review has many well understood benefits beyond its ability to find bugs, including learning and mentoring, (collective) code ownership, and general awareness – not captured in the simplified model (P, L, E, R). Pushing more code through the review process or automating code review away will obviously degrade those benefits too: If developers review less, they may gain less experience that helps them in producing code or being effective at those reviews still remaining. Developers may not understand their code base at all, making it harder for them to write good prompts to change it further, possibly increasing liability indirectly through cognitive debt. Or tooling can cover enough of what is lost, freeing up developers' time to focus on other parts that matter (e.g., requirements, specifications for test generation, infrastructure for testing in production).

Such effects are all plausible, but it is not clear to me how it will shake out. If we want to argue for code review, we should be clear about what benefits we want to gain from it (beyond finding bugs). And frankly, we should look for ways of getting those benefits without traditional code review. More research is needed, and in it we must be explicit about the mechanisms we expect and what effects we predict, whether on production cost, review cost, liability, or anything else.

Code review changed before

[↘ liability due to cheaper review substitutes] Third, alternatives to code review get cheaper too: Tests, static analysis, and automated code review, and maybe one day even formal verification and proof-carrying code at scale, can catch problems early during code production, reducing liability before code review. If such activities become effective and cheap enough, they render code review less important for liability reduction.

I do not think it is clear yet how this will pan out. In low-stakes settings, self-reviewing agents, more testing, and cheap rework may seriously reduce most liability, but it seems less obvious how it will affect the kind of downstream harms that matter in high-stakes settings – security issues, safety issues, financial losses. When issues can create real harms, patching a system in production after harms have occurred may not be the best strategy.

Productivity pressure, surface plausibility of agentic code, and cognitive debt can each undermine review effectiveness (E). [↘ effectiveness due to productivity pressure] Many practitioners report enormous pressure to improve productivity with AI. With more code arriving from agents and review becoming the bottleneck, this translates into pressure to review faster, which can result in rubber-stamping, undermining any effectiveness modern code review may still have had.

[↘ effectiveness due to deceptive plausibility of AI generated code and cognitive debt] In addition, agentic code may be harder to review well, because it looks plausible and well written (automation bias), and because a reviewer who was not engaged in producing it often lacks the context to judge it (cognitive debt). Both issues lower review effectiveness, or would require more investment into code review to retain effectiveness.

These effects make preserving traditional human code review particularly challenging if most code is generated by AI. Insisting on manual review, but pushing more reviews through faster, likely just hollows it out. I fully agree that this will not be sustainable and code review will not survive in this form.

Automating the code review may lower review cost (R). [↘ review cost with review automation] Many of the quality problems that surface in modern code review, especially the shallow ones modern code review catches, can be found by machines mimicking code review. *Automated code review* by agents can arguably find many of those kinds of issues and lower review costs similar to how AI lowered production costs. Some offerings of automated code review can be quite pricey (e.g., [Claude’s code review costs \\$15-25 per reviewed PR](#)), but they still likely undercut the costs of humans and they don’t share the human capacity bottleneck.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

A load-bearing poster.

"Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentially-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
❌❌❌❌❌❌ Failed Checks: 1 ❌❌❌❌❌❌
```

```
FAIL src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a
```

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:

- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:

- remove duplicated rendering paths or dead production code
- move repeated styling or static configuration out of lazy-loaded Ja
- lazy-load rarely used flows instead of loading them on the main pat (more examples here)

Do not fix this by:

- omitting requested scope or deferring needed UI work
- removing aria-label/title text or visible affordances
- deleting helpful empty/error-state copy (more examples here)

For intentional growth, (procedure here) and explain why in the PR.

Agentic coding shifts every part of the model, all at once

Agentic coding clearly has the potential to lower the cost of code production, but it can also influence the cost and effectiveness of code review, and the liability of typical code produced. The specific effects are often disputed, point in different directions, and interact with various things changing at the same time. The model can help us to untangle what we might expect to happen.

Production cost (P) is the obvious starting point but only part of the story. [↘ production cost] Agentic coding promises to drastically cut the cost of producing code (P). If nothing else changed (e.g., same code review, agentic code producing the same liability as human code), review cost (R) and liability cost (L) limit how much we can reduce overall cost. For example, even if production costs are reduced by 90%, overall costs would just be reduced by 30% if production accounted for only a third of the costs previously. This effect likely explains some of the discrepancies between frequent claims of 10X productivity improvement (which may even feel intuitively right seeing how quickly code is produced) and much lower productivity improvements measured in academic studies (rarely above 30%). If production cost is the only thing that changes, code review feels very much like a bottleneck limiting progress.

Agentic coding affects liability (L) in at least three plausible ways, and it is not obvious where we will land. When it comes to code quality and liability, the effect of coding agents is disputed.

[↗ liability due to lower code

There may be many plausible reasons for the almost universal adoption of modern code reviews – whether companies copying successful open source practices (coined “inner source”) or just wanting to be more like Google, whether inertia, or whether developers just do not trust their interns – but let’s assume somewhat rational actors: If modern code review wouldn’t have provided any benefits, we would have stopped spending so much effort on it. For the sake of this argument, let’s assume, the community has eventually settled on an **equilibrium** where the value modern code review provides just justified its cost in a pre-AI world, whether through catching bugs or through those other benefits.

A simple model to untangle competing arguments

Before looking at how agentic coding puts code review under pressure, let’s posit a simple model that will help to untangle various moving parts to help put various forces into perspective.

- **P**: The cost of producing code, in \$ per line.
- **L**: The liability of that code, in \$ per line, meaning the expected future cost of its quality problems. Let’s assume liability has two parts: (a) **Rework**, the cost of finding and fixing bugs or in the worst case rearchitect the entire system. (b) **Harm**, the cost when problems escape to the outside world as security incidents, reputational damage, and losses to customers we may need to reimburse, sometimes after lawsuits. Rework is internal and is the main concern for low-stakes software, but harm is visible externally and can be substantial in high-stakes software.
- **R**: The cost of reviewing code, in \$ per line.
- **E**: The effectiveness of review at reducing liability, as a percentage.

Overall cost per line is then $P + L \times (1 - E) + R$. In a rational-actor world, teams invest in code review exactly up to the point where the last dollar spent on it just offsets the liability it removes.

For high-stakes code with high potential liability (L), it is rational to invest heavily (R) into making review more effective (E), up to Fagan-style inspections. For most code, stakes are not that high and liability comes mostly from rework. Also many other efforts have helped reduce liability without code review: Modern languages prevent whole classes of memory defects, static analysis catches many issues, and test generation and continuous-integration automation move testing into the cost of production. In that setting, shallow and fairly ineffective modern code review looks rational with modest cost for modest liability reduction.

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

A master bedroom.

"Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.

$$C = P + L \times (1 - E) + R$$

The diagram illustrates the equation $C = P + L \times (1 - E) + R$ with the following annotations:

- C**: Total cost (in \$/LOC)
- P**: Production cost (in \$/LOC)
 - ↳ Agents produce code faster
- L**: Liability (in \$/LOC): Rework + Harm
 - ↳ Lower quality, more harm and rework
 - ↳ Higher quality, less harm and rework
 - ↳ Agents lowering rework costs
 - ↳ Cheaper substitutes: tests, proofs
- E**: Effectiveness of Reviews (in %)
 - ↳ Productivity pressure
 - ↳ Deceptive plausibility
 - ↳ Cognitive debt
- R**: Review cost (in \$/LOC)
 - ↳ Automated code review

A sweet spot for code review, historically?

Code review did not always look the way it does now (or at least the way it looked until coding agents took off). The contemporary practice that we now all know – tool-mediated, CI-integrated, lightweight review of individual commits, usually integrated with testing, static analysis, and other automation – is what reviewers call **modern code review** to distinguish it from other forms of code review and inspection.

Modern code review is called *modern* in contrast to older practices that many junior engineers have probably never seen and that we barely teach these days, most popular among them *Fagan-style inspections*. These were heavyweight processes, popularized in the 70s and widely studied through the 80s and early 90s: Multiple reviewers read small fragments of code line by line before release (not incrementally while code is created

↵

3. In theory, at least. In practice it's always better to be useful (again, in the sense of “delivering shareholder value”).

↵

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This [draft](#) is the closest thing I have to a deeper exploration of the point.

↵

5. For the record, my how-to guides are [here](#) and [here](#).

↵

The End of Code Review? Or an Opportunity to Rethink it?

CHRISTIAN KÄSTNER · 26 SEP 2026 · [SOURCE](#)

Code review as we understand it today will probably not survive the shift to AI-generated code, but is that a problem? Code review is under pressure from agents producing code at an incredible pace, and reactions to that are quite split. There are many who want to protect code review as a core human software engineering practice, who put in place strict policies to preserve it. Others argue that code review is effectively dead, because it cannot keep up with the pace of production and will not survive competitive pressures if it becomes a bottleneck. Many experiment with automating code review at least partially. Implicit in this discussion is often the assumption that our prior code review practices were a gold standard, something to save or something to mourn. But is that really the case?

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · [SOURCE](#)

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's [Staff engineer archetypes](#). He argues that the “staff engineer” title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For [both](#) of my promotions to staff engineer, my manager at the time linked me to the “staff engineer archetypes” and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist¹. However, I think it's bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let's take the “team lead” archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully [ship](#). If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard “to be a writer”. You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in [Ratchet effects determine engineer reputation at large companies](#). To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly larger pieces. At each stage, if you do good work - “good work” here means “deliver [shareholder value](#)” - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you're going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you'll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with

- You likely won't have a good enough relationship with senior management to know what their real priorities are.
- If you're not yet trusted to execute, you may get assigned "minders" (often current staff engineers) who will ghost-lead the project through you².
- You'll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can't design software you don't work on. The "solver" and "right hand" archetypes both rely on having an enormous amount of trust and influence. You can't aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of "aiming for" a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don't get the support needed to succeed) that's their manager's problem, not theirs³. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can't be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn't possible to conjure useful work from thin air. That's actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don't realize is that all senior management and executive leadership roles are unfair too, in the same way. That's just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather⁴. "Staff engineer" is the first engineering role where you are held largely responsible for outcomes you don't control.

Developing a "staff engineer mindset" thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself "is this useful to the company" (and answering correctly).

2. Lose the habit of worrying about if you're being treated "fairly". Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won't look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You'll also look like someone who's paying a lot of attention to what their manager's actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you'll eventually find yourself in one of the staff engineer archetypes. However, it probably won't be the one you're "aiming for". The whole point of being a staff engineer is that you're willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role⁵. At the time, the "staff engineer" role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they've built a deep relationship of trust with their organization's leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.